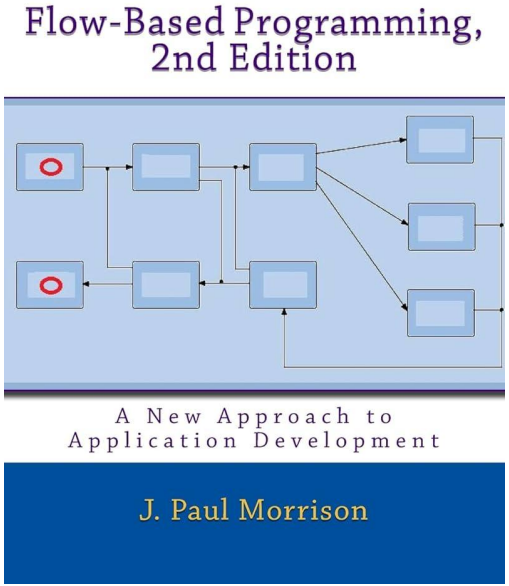
FBP / DES / SD

or how to not write an elevator simulator

- Networks of independent black boxes
- Asynchronous communication
- No shared state
- Predefined connections
- Discovered in the early 1970s



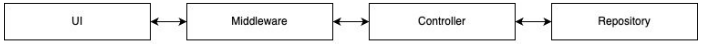
Limitations in practice:

- Purely visual or no-code programming is not practical
- Simple design does not emerge automatically

AWS 3-tier



n-tier



Strict separation of concerns

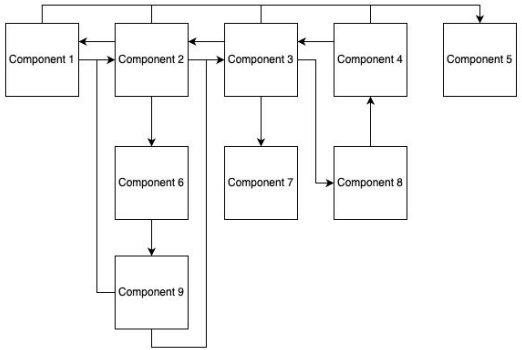
Convention-driven structure

Control-flow oriented

Spring Boot Layered architecture

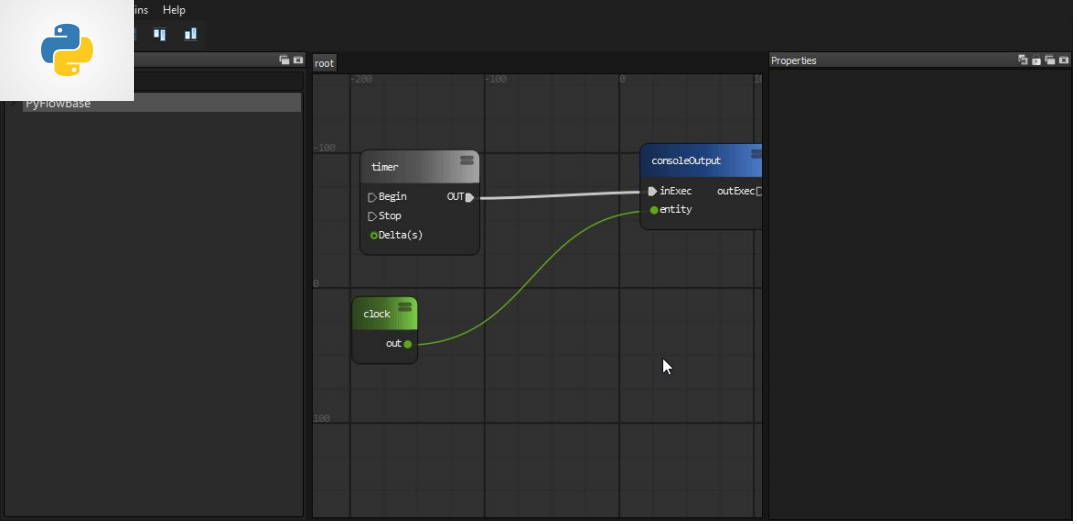


FBP



No fixed layers or call hierarchy *(you can have it when needed)*

Data-driven, not control-driven



APACHE nifi

Java

Ryven

JS Node-RED

JS NoFlo

FRACTALIDE

ALF

elixir

hex v0.12.1

Flow-based Application Layer Framework

Anything in Go?

Plain Go

- Just goroutines and channels

Flowbase

- Lightweight abstraction over Go channels
- ~40 commits – small and actively maintained

Goflow

- Components connected explicitly via channels
- ~200 commits – last update 4 years ago

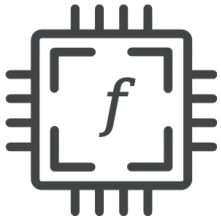
Cascades

- Language-agnostic FBP runtime with NoFlo DSL
- Last update 11 years ago

Red flags !



```go
func main() {
    // Create the network
    net := NewGreetingApp()
    // We need a channel to talk to it
    in := make(chan string)
    net.SetInPort("In", in)
    // Run the net
    wait := goflow.Run(net)
    // Now we can send some names and see what happens
    in <- "John"
    in <- "Boris"
    in <- "Hanna"
    // Send end of input
    close(in)
    // Wait until the net has completed its job
    <-wait
}
```



FMesh - FBP inspired framework in Go

Components are functions with optional internal **state**

Type-agnostic signals, ports, and pipes

Discrete time via activation cycles

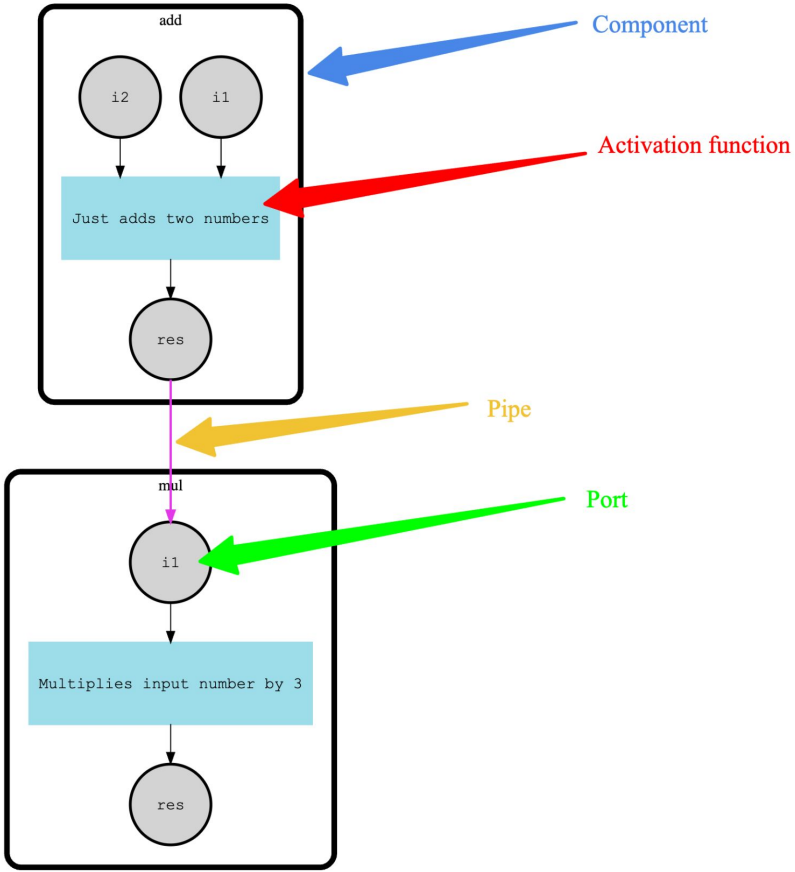
Visualisation

Built in **fan-in / fan-out**

Concurrency by default

Zero dependencies

Hello mesh !


add
i2
i1
Just adds two numbers
res
Component
Activation function
Pipe
mul
i1
Multiplies input number by 3
res
Port

```
adder := component.New("add").
    WithDescription("Just adds two numbers").
    WithInputs("i1", "i2").
    WithOutputs("res").
    WithActivationFunc(func(this *component.Component) error {
        num1 := this.InputByName("i1").FirstSignalPayloadOrDefault(0).(int)
        num2 := this.InputByName("i2").FirstSignalPayloadOrDefault(0).(int)

        this.OutputByName("res").PutSignals(signal.New(num1 + num2))
        return nil
    })
```

```
multiplier := component.New("mul").
    WithDescription("Multiplies input number by 3").
    WithInputs("i1").
    WithOutputs("res").
    WithActivationFunc(func(this *component.Component) error {
        num1 := this.InputByName("i1").FirstSignalPayloadOrDefault(0).(int)

        this.OutputByName("res").PutSignals(signal.New(num1 * 3))
        return nil
    })
```

```
// Create the mesh (graph)
fm := fmesh.New("my first mesh").WithComponents(adder, multiplier)

// Connect pipes
adder.OutputByName("res").PipeTo(multiplier.InputByName("i1"))

// Init inputs
adder.InputByName("i1").PutSignals(signal.New(10))
adder.InputByName("i2").PutSignals(signal.New(20))
```

```go
// Run the mesh
_, err := fm.Run()

// Check for errors
if err != nil {
    fmt.Println("F-Mesh returned an error")
    os.Exit(1)
}

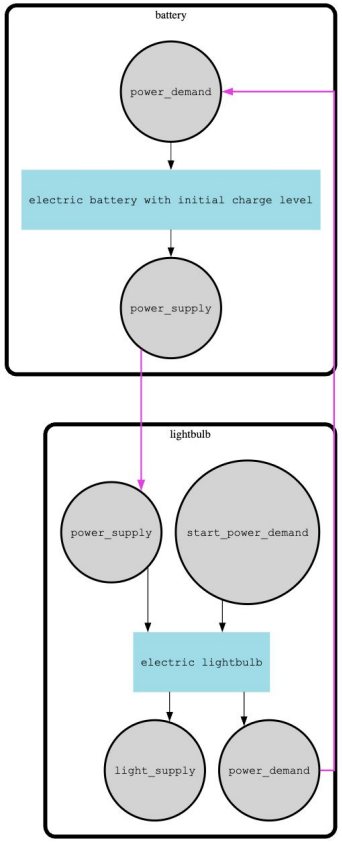
// Extract results
result := multiplier.OutputByName("res").FirstSignalPayloadOrNil()
fmt.Printf("Result is : %v", result)
```

```
Result is : 90
Process finished with the exit code 0
```

Why use graph?

```
func main() {
    a := 10
    b:= 20

    fmt.Printf("Result is : %v", (a+b)*3)
}
```




```go
battery := component.New("battery").
    WithDescription("electric battery with initial charge level").
    WithInputs("power_demand").
    WithOutputs("power_supply").
    WithInitialState(func(state component.State) {
        state.Set("level", 1000)
    }).
    WithActivationFunc(func(this *component.Component) error {
        // Read state
        level := this.State().Get("level").(int)

        defer func() {
            // Write state
            this.State().Set("level", level)
        }()

        this.Logger().Println("level: ", level)
        // Power demand/supply cycle
        if this.InputByName("power_demand").HasSignals() {
            demandedCurrent := this.InputByName("power_demand").FirstSignalPayloadOrDefault(0).(int)

            // Emit current represented as a number
            suppliedCurrent := min(level, demandedCurrent)
            if suppliedCurrent > 0 {
                this.OutputByName("power_supply").PutSignals(signal.New(suppliedCurrent))
                this.Logger().Println("supplying power:", suppliedCurrent)

                if suppliedCurrent < demandedCurrent {
                    this.Logger().Println("LOW BATTERY")
                }

                // Discharge
                level = max(0, level-suppliedCurrent)
                this.Logger().Println("discharged to: ", level)
            } else {
                this.Logger().Println("BATTERY DIED")
            }
        }

        return nil
    })
```

```go
// Read state
temperature := this.State().Get("temperature").(float64)

defer func() {
    // Write state
    this.State().Set("temperature", temperature)
}()

// Skip power consumption on start (as power is not demanded yet)
if !this.InputByName("start_power_demand").HasSignals() {
    // Power consumption cycle (at constant rate)
    inputPower := this.InputByName("power_supply").FirstSignalPayloadOrDefault(0).(int)
    this.Logger().Println("got power: ", inputPower)

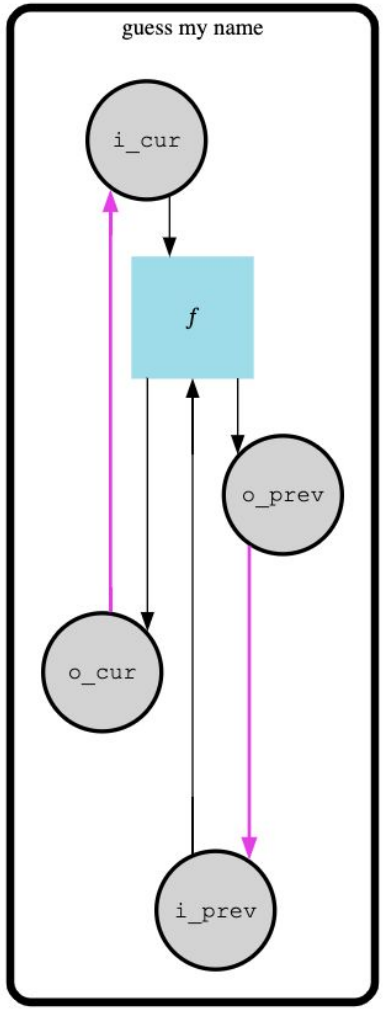
    if inputPower >= lightBulbPowerConsumption {
        // Emit light (here we simulate how the lightbulb performance degrades with warming)
        lightEmission := lightBulbLuminousFlux / temperature * 100
        if temperature >= lightbulbOverheatingThreshold {
            this.Logger().Println("OVERHEATING. LIGHT EMISSION WILL SIGNIFICANTLY DEGRADE")
            lightEmission *= lightbulbOverheatDegradation
        }
        this.OutputByName("light_supply").PutSignals(signal.New(lightEmission))
        this.Logger().Println("emitted light: ", lightEmission)
        this.Logger().Println("temperature:", temperature)

        // Get warmer
        temperature += lightBulbWarmingPerCycle

        if temperature > lightbulbMaxWorkingTemperature {
            this.Logger().Println("BURNOUT")
            return nil
        }
    } else {
        this.Logger().Println("POWER STARVATION")
    }
}

// Always continue demanding power
this.OutputByName("power_demand").PutSignals(signal.New(lightBulbPowerConsumption))
return nil
```

Guess the algo !


guess my name
i_cur
f
o_prev
o_cur
i_prev

```
generator := component.New("guess my name").
    WithInputs("i_cur", "i_prev").  // Hint: current and previous numbers in the sequence
    WithOutputs("o_cur", "o_prev"). // Hint: next and previous numbers in the sequence
    WithActivationFunc(func(this *component.Component) error {
        cur := this.InputByName("i_cur").FirstSignalPayloadOrDefault(0).(int)
        prev := this.InputByName("i_prev").FirstSignalPayloadOrDefault(0).(int)

        next := cur + prev

        // Hardcoded limit
        if next < 100 {
            fmt.Println(next) // Hint: print the next number in the sequence
            this.OutputByName("o_cur").PutSignals(signal.New(next))
            this.OutputByName("o_prev").PutSignals(signal.New(cur))
        }
        return nil
    })

// Define pipes
generator.Outputs().ByName("o_cur").PipeTo(generator.Inputs().ByName("i_cur"))
generator.Outputs().ByName("o_prev").PipeTo(generator.Inputs().ByName("i_prev"))

// Build mesh
fm := fmesh.New("example mesh").WithComponents(generator)

// Hint: set inputs (first 2 numbers)
generator.Inputs().ByName("i_prev").PutSignals(signal.New(0))
generator.Inputs().ByName("i_cur").PutSignals(signal.New(1))

fmt.Println("0")
fmt.Println("1")

// Run the mesh
_, err := fm.Run()
```

Right answer:

Fibonacci sequence generator

```
0
1
1
2
3
5
8
13
21
34
55
89
```

Example: CAN-BUS simulation



Use cases & limitations

Computation graph

Simulations

Control flow, state machines

Discrete time
(no tickers and timers)

Strict concurrency model
(slow component problem)

No ready to use components
(contributors are welcome!)

Computation graph

Represent any statement or equation as a graph:

Nodes: operations (e.g., add, multiply)

Edges: data dependencies (flow of values)

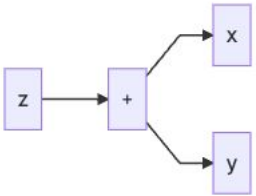
Key Properties:

Traversability: easy to walk through computations step by step

Automatic Differentiation: compute derivatives efficiently for optimization

Perfect for ML & Deep Learning

The equation $x + y = z$ can be represented as a graph:



GORGONIA

```
func main() {
        g := gorgonia.NewGraph()

        var x, y, z *gorgonia.Node
        var err error

        // define the expression
        x = gorgonia.NewScalar(g, gorgonia.Float64, gorgonia.WithName("x"))
        y = gorgonia.NewScalar(g, gorgonia.Float64, gorgonia.WithName("y"))
        if z, err = gorgonia.Add(x, y); err != nil {
                log.Fatal(err)
        }

        // create a VM to run the program on
        machine := gorgonia.NewTapeMachine(g)
        defer machine.Close()

        // set initial values then run
        gorgonia.Let(x, 2.0)
        gorgonia.Let(y, 2.5)
        if err = machine.RunAll(); err != nil {
                log.Fatal(err)
        }

        fmt.Printf("%v", z.Value())
}
```

Autodiff example

```go
import (
    "fmt"
    "log"

    G "gorgonia.org/gorgonia"
)

// export ASSUME_NO_MOVING_GC_UNSAFE_RISK_IT_WITH=go1.25
func main() {
    // Create a new computation graph
    g := G.NewGraph()

    // Define scalar nodes x and y
    x := G.NewScalar(g, G.Float64, G.WithValue(1.0), G.WithName("x"))
    y := G.NewScalar(g, G.Float64, G.WithValue(1.0), G.WithName("y"))

    // z = x + y
    z := G.Must(G.Add(x, y))

    // Ask Gorgonia to compute gradients of z with respect to x and y
    if _, err := G.Grad(z, x, y); err != nil {
        log.Fatal("Failed to compute gradient:", err)
    }

    // Create a VM (TapeMachine) to execute the graph
    vm := G.NewTapeMachine(g)
    defer vm.Close()

    // Run the forward + backward passes
    if err := vm.RunAll(); err != nil {
        log.Fatal("Error running the VM:", err)
    }

    // Print forward result
    fmt.Printf("z = %v\n", z.Value()) // Output: z = 2

    // Get gradients safely
    xGrad, err := x.Grad()
    if err != nil {
        log.Fatal("x.Grad() error:", err)
    }
    yGrad, err := y.Grad()
    if err != nil {
        log.Fatal("y.Grad() error:", err)
    }

    // Print gradients
    fmt.Printf("dz/dx = %v\n", xGrad) // Output: dz/dx = 1
    fmt.Printf("dz/dy = %v\n", yGrad) // Output: dz/dy = 1
}
```

Simulation models

DES (Discrete Event Simulation) – event-driven process modeling

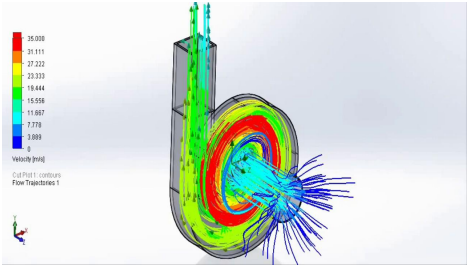
SD (System Dynamics) – continuous feedback and stock-flow systems

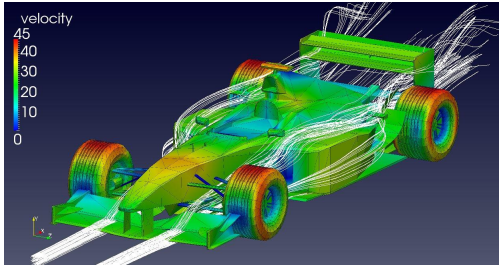
ABM (Agent-Based Modeling) – individual entities with behaviors

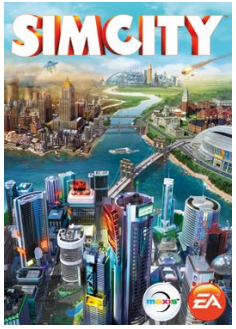
MC (Monte Carlo) – probabilistic sampling and uncertainty modeling

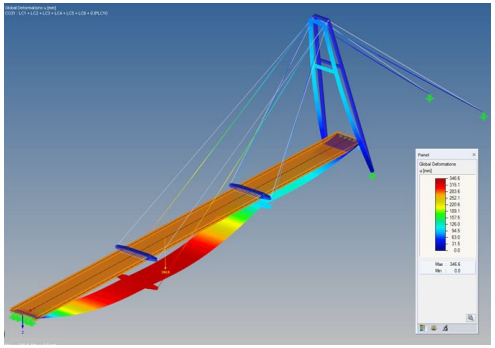
Petri Nets – concurrency and process coordination

Cellular Automata – grid-based evolution of local rules









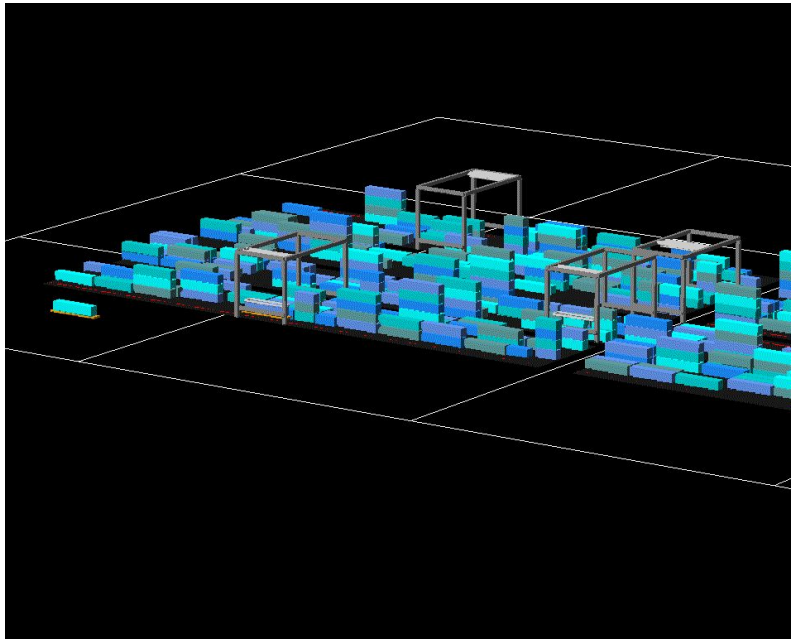

what is AnyLogic

DES

Jumps between discrete events; time between events is ignored.

State updates only at events: next_state = f(state).

Simple and efficient for gaming, manufacturing, logistics.



Libraries by Language

Python:

SimPy - de facto standard for discrete event simulation.

Go:

Godes, SimGo, Steps - useful libraries, but no one dominates the ecosystem.

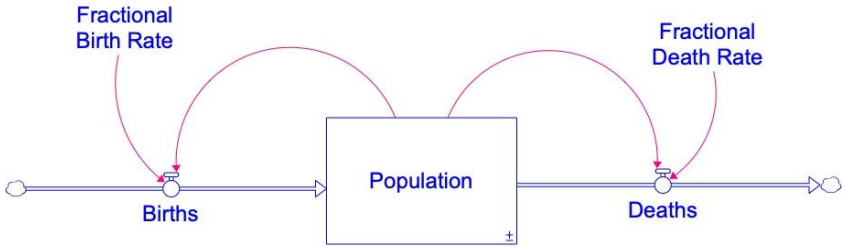
System Dynamics

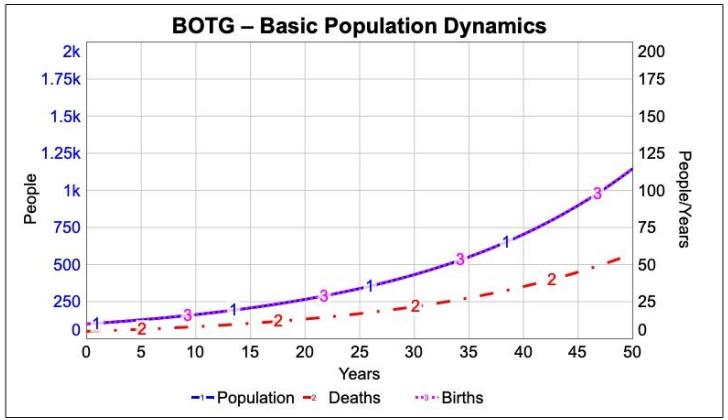
Continuous change over time:
Models how the **system evolves** rather than jumping between discrete events.

Mathematical foundation:
Uses **differential equations** and numerical **integrators** to simulate behavior.

Example applications:

- Physics: air or fluid flow, rigid body dynamics
- Economics: money and resource flows
- Biology & Medicine: population dynamics, disease spread





SD libraries

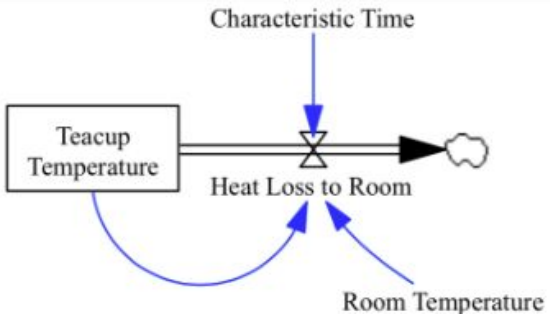
Python:

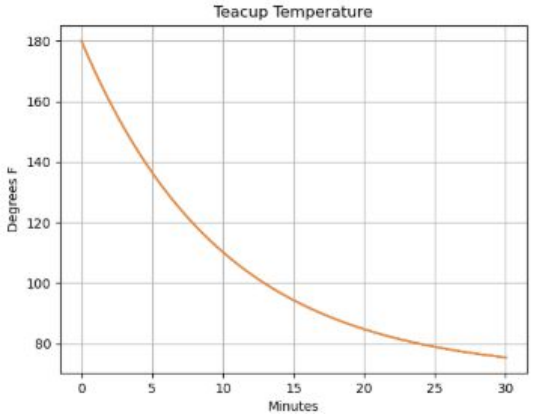
PySD, BPTK-Py — widely used, full-featured SD modeling libraries.

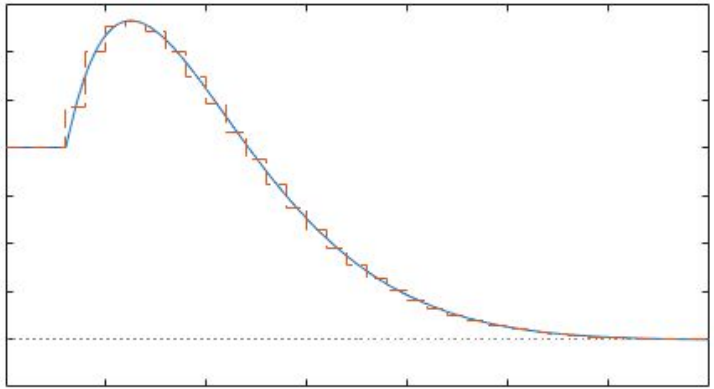
Go:

No dedicated SD libraries.

Can approximate SD using DES libraries by reducing the timestep (Δt) and updating stocks at each event.







Summing up

Flow-Based Programming (FBP): programs as graphs of data and computation

Graphs enable:

- **ML** - computation graphs for differentiable learning
- **DES** - event flow and discrete simulation
- **SD** - feedback loops and system dynamics

Go **needs** stronger FBP and simulation libraries - ***let's catch up with Python!***

Thank you!

